Data Structure Lab8 : Queue 2022-2023

Topics

1. Create Queue Interface
2. Create Queue Using Array
3. Create Queue Using Linked Lists
4. Implement Basic Methods of Queue
    - isEmpty()
    - size()
    - first()
    - enqueue(E e)
    - dequeue()

Homework

1. Augment the ArrayQueue implementation with a new rotate( ) method having semantics identical to the combination, enqueue(dequeue( )). But, your implementation should be more efficient than making two separate calls (for example, because there is no need to modify the size).

    ```
    public void rotate() {
        if (isEmpty()) return; // No rotation needed
        E element = dequeue(); // Remove the front element
        enqueue(element); // Enqueue it at the back
    }
    ```

2. Implement the clone( ) method for the ArrayQueue class.

    ```
    public ArrayQueue<E> clone() {
        ArrayQueue<E> newQueue = new ArrayQueue<>(this.size());
        for (int i = 0; i < this.size(); i++) {
    ```

```
            newQueue.enqueue(this.array[(front + i) % array.length]); //
    Copy elements
        }
        return newQueue;
    }
```

3. Implement a method with signature concatenate(LinkedQueue Q2) for the LinkedQueue class that takes all elements of Q2 and appends them to the end of the original queue. The operation should run in O(1) time and should result in Q2 being an empty queue.

```
public void concatenate(LinkedQueue<E> Q2) {
    if (Q2.isEmpty()) return;
    if (this.isEmpty()) {
        this.front = Q2.front;
        this.rear = Q2.rear;
    } else {
        this.rear.next = Q2.front;
        this.rear = Q2.rear;
    }
    Q2.front = null;
    Q2.rear = null;
}
```

4. Use a queue to solve the Josephus Problem.

```
public static int josephus(int n, int k) {
    Queue<Integer> queue = new LinkedQueue<>();
    for (int i = 1; i <= n; i++) {
        queue.enqueue(i); // Enqueue people
    }

    while (queue.size() > 1) {
```

```
        for (int i = 0; i < k - 1; i++) {
            queue.enqueue(queue.dequeue()); // Rotate to the k-th
person
        }
        queue.dequeue(); // Remove the k-th person
    }
    return queue.dequeue(); // The last remaining person
}
```

5. Use a queue to simulate Round Robin Scheduling.

```
public void roundRobinScheduling(Queue<Process> queue, int
timeSlice) {
    while (!queue.isEmpty()) {
        Process current = queue.dequeue();
        if (current.getRemainingTime() > timeSlice) {
            current.reduceTime(timeSlice);
            queue.enqueue(current);
        } else {
            current.complete();
        }
    }
}
```